# GPU-Batched Non-Homogeneous Heat-Equation Reconstruction (Per-$t_k$ Approach)

## 1 Problem Setting

We study the 1D heat equation with Dirichlet boundary conditions on $(0, L)$ and a source (forcing) term:

$$\begin{cases} u_t(x,t) - u_{xx}(x,t) = F(x,t), & x \in (0,L),\ t > 0, \\ u(0,t) = u(L,t) = 0, & t > 0, \\ u(x,0) = f(x), & x \in (0,L). \end{cases}$$

The goal of the reconstruction algorithms in this codebase is:

- Given measurements of $u(x_0, t_k)$ at a single spatial point $x_0$ and a sequence of times $t_1, \ldots, t_n$, reconstruct the initial condition $f(x)$ (or an approximation $\bar{f}_n(x)$) via a truncated sine series.

The homogeneous case $F \equiv 0$ yields a simpler reconstruction; the non-homogeneous case requires separating the contributions of the initial condition and the forcing term.
Throughout, for $L = \pi$, the eigenfunctions and eigenvalues are:

$$\phi_j(x) = \sin(jx), \quad \lambda_j = j^2, \quad j = 1, 2, \ldots$$

and we expand

$$f(x) = \sum_{j=1}^{\infty} \hat{f}_j \sin(jx),$$

with Fourier-sine coefficients

$$\hat{f}_j = \frac{2}{L} \int_0^L f(x) \sin\Big(jx\Big)\, dx.$$

In the code, $L$ may be general, in which case $\phi_j(x) = \sin(j\pi x/L)$, $\lambda_j = (j\pi/L)^2$, and the formulas adjust accordingly.

# 2 Homogeneous Reconstruction at a Single Point

## 2.1 Forward model

In the homogeneous case $F \equiv 0$, the solution at $(x_0, t_k)$ is

$$u(x_0, t_k) = \sum_{j=1}^{\infty} \hat{f}_j \, \mathrm{e}^{-\lambda_j t_k} \sin(j x_0).$$

Truncating at $j \le n_{\text{terms}}$ and denoting

$$u_k := u(x_0, t_k), \qquad v_{k,j} := \mathrm{e}^{-\lambda_j t_k} \sin(j x_0),$$

we have approximately

$$u_k \approx \sum_{j=1}^{n_{\text{terms}}} \hat{f}_j \, v_{k,j}.$$

In matrix form:

$$u_k \approx V \hat{f}_j,$$

where $u \in \mathbb{R}^n$, $\hat{f}_j \in \mathbb{R}^{n_{\text{terms}}}$, and $V \in \mathbb{R}^{n \times n_{\text{terms}}}$.
The "modified" $t_j$ reconstruction used in this project does not invert the entire matrix $V$ at once, but instead uses a carefully chosen time sequence and a recursive scheme for $\bar{f}_{\hat{k}}$ that is numerically more stable and tailored to the measurement sequence.

## 2.2 Recursive reconstruction (homogeneous part)

For a given time vector $t_k$ (e.g. the modified factorial sequence or a linear sequence), the code computes:

- A synthetic data vector $u_{\text{data},k}$ approximating $u(x_0, t_k)$.
- A sequence of reconstructed Fourier coefficients $\bar{f}_{\hat{k}}$ for $k = 1, \ldots, \lceil n/2 \rceil$.

The key recurrence (in the homogeneous case) is:

$$\begin{aligned} u_k &= \sum_{j=1}^{k-1} \mathrm{e}^{-\lambda_j t_k} \bar{\hat{f}}_j \sin(j x_0) \\ &\quad + \mathrm{e}^{-\lambda_k t_k} \bar{f}_{\hat{k}} \sin(k x_0), \\ \Rightarrow \quad u_k - \sum_{j=1}^{k-1} \mathrm{e}^{-\lambda_j t_k} \bar{\hat{f}}_j \sin(j x_0) &= \mathrm{e}^{-\lambda_k t_k} \bar{f}_{\hat{k}} \sin(k x_0), \\ \bar{f}_{\hat{k}} &= \mathrm{e}^{\lambda_k t_k} \, \frac{u_k - \sum_{j=1}^{k-1} \mathrm{e}^{-\lambda_j t_k} \bar{\hat{f}}_j \sin(j x_0)}{\sin(k x_0)}. \end{aligned}$$

For $k = 1$, the sum is empty, so

$$\bar{f}_{\hat{1}} = \mathrm{e}^{\lambda_1 t_1} \, \frac{u_1}{\sin(x_0)}.$$

This is precisely what the reconstruction loops implement (both in the NumPy and CuPy versions), with various precomputations to avoid redundant work.
Once $\bar{f}_{\hat{k}}$ are computed up to $k = \lceil n/2 \rceil$, the reconstructed initial condition is

$$\bar{f}_n(x) = \sum_{k=1}^{\lceil n/2 \rceil} \bar{f}_{\hat{k}} \, \sin(kx).$$

# 3 Non-Homogeneous Case: Decomposition and Truncation

In the non-homogeneous case, the solution decomposes as

$$u(x,t) = u^{(0)}(x,t) + u^{(F)}(x,t),$$

where:

- $u^{(0)}(x,t)$ solves the homogeneous equation with initial data $f(x)$,
- $u^{(F)}(x,t)$ accounts for the forcing.

## 3.1 Forcing coefficients and solution structure

Expand the forcing $F(x,t)$ in the same sine basis:

$$F(x,t) = \sum_{j=1}^{\infty} \hat{f}_j(t) \, \sin(jx),$$

with

$$\hat{f}_j(t) = \frac{2}{L} \int_0^L F(x,t) \, \sin\Big(jx\Big) \, dx.$$

The contribution of the forcing to the solution at a point $x_0$ and time $t_k$ is

$$u^{(F)}(x_0,t_k) = \sum_{j=1}^{\infty} \left( \int_0^{t_k} \hat{f}_j(s) \, \mathrm{e}^{-\lambda_j (t_k - s)} \, ds \right) \sin(jx_0).$$

Define the (time-dependent) modal response:

$$I_j(t_k) := \int_0^{t_k} \hat{f}_j(s) \, \mathrm{e}^{-\lambda_j (t_k - s)} \, ds.$$

Then

$$u^{(F)}(x_0,t_k) = \sum_{j=1}^{\infty} I_j(t_k) \, \sin(jx_0).$$

### 3.2 Truncation index $n_k$ and the constant $C$

The series over $j$ is truncated at a (possibly time-dependent) index $n_k$, chosen so that the tail is negligible relative to the reconstruction error. The truncation index is defined via a constant $C$ that depends on the spatial derivative $F_x$:

$$C = \frac{2}{L} \max_{s\in[0,T]} \|F_x(\cdot, s)\|_{L^1(0,L)} .$$

Then

$$n_k = \left\lceil C\, \mathrm{e}^{\frac{(k+1)^2 t_k}{2}} \right\rceil ,$$

as implemented in `compute_n_k`. Intuitively, modes with index $j > n_k$ contribute very little due to the exponential decay of the heat kernel and the decay of the Fourier coefficients controlled by $C$.
For the default forcing in the CLI demo,

$$F(x,t) = \sin(x)\, \mathrm{e}^{-t}, \quad L = \pi,$$

we have

$$F_x(x,s) = \cos(x)\, \mathrm{e}^{-s}.$$

The $L^1$-norm is

$$\|F_x(\cdot, s)\|_{L^1(0,\pi)} = \int_0^\pi |\cos(x)|\, \mathrm{e}^{-s}\, dx = \mathrm{e}^{-s} \int_0^\pi |\cos(x)|\, dx = \mathrm{e}^{-s} \cdot 2,$$

and since $\mathrm{e}^{-s}$ decreases in $s$, the maximum over $s \in [0, T]$ is attained at $s = 0$:

$$\max_{s\in[0,T]} \|F_x(\cdot, s)\|_{L^1(0,\pi)} = 2.$$

Thus, for this forcing,

$$C = \frac{2}{\pi} \cdot 2 = \frac{4}{\pi}.$$

This is encoded and tested in `compute_forcing_truncation_constant_C`.

# 4 CPU Algorithm vs GPU-Batched Algorithm

## 4.1 CPU forcing algorithm (high-level)

In `forcing.py`, the CPU algorithm approximates $I_j(t_k)$ using a two-strategy method:

1. Low-frequency modes $j = 1, \ldots, N$ (full integral path):

   - Precompute $\hat{f}_j(s)$ on a time grid $s_0, \ldots, s_{n_s-1} \subset [0, t_k]$ via fixed-order Gauss–Legendre quadrature in $x$.

- Integrate in $s$ with the kernel $\mathrm{e}^{-\lambda_j(t_k-s)}$ using the trapezoidal rule.

2. High-frequency modes $j = N+1, \ldots, n_k$ (asymptotic path):
   - Use the approximation
   $$I_j(t_k) \approx \frac{\hat{f}_j(t_k)}{\lambda_j},$$
   which is valid when $\mathrm{e}^{-\lambda_j t_k}$ is already very small.

This is implemented in `compute_forcing_contribution_at_time`, with the cutoff $N$ determined by `compute_exp_decay_cutoff`.
Although numerically robust, this approach is CPU-heavy: it nests quadrature over $x$ inside loops over $j$ and $s$, and then loops over time steps $k$. This is the main source of long runtimes in the non-homogeneous demos.

## 4.2 Finding the Cutoff for Exponential

The forcing contribution to $u(x_0, t_k)$ has the form (for each mode $j$):

$$\sum_{j=1}^{n_k} \underbrace{\left( \int_0^{t_k} \hat{F}_j(s)\, e^{-\lambda_j(t_k-s)}\, \mathrm{d}s \right)}_{\text{time integral}} \sin(jx_0), \qquad \lambda_j = \left(\tfrac{j\pi}{L}\right)^2.$$

For high $j$, $\lambda_j$ is large, so $e^{-\lambda_j(t_k-s)}$ is tiny except when $s$ is very close to $t_k$. Over most of $[0, t_k]$ the kernel is negligible, so the time integral is well approximated by a simple formula that doesn't need quadrature: effectively $\hat{F}_j(t_k)/\lambda_j$ (the "asymptotic" approximation).

The code uses the full time integral only when that exponential is not negligible. So we want:

- Modes $j \le N$: $e^{-\lambda_j t_k}$ is not negligible $\to$ use the full integral (quadrature).
- Modes $j > N$: $e^{-\lambda_j t_k}$ is below a tolerance $\to$ use the cheap approximation $\hat{F}_j(t_k)/\lambda_j$.

The cutoff $N$ is chosen so that for $j > N$ we have $e^{-\lambda_j t_k} < \texttt{tol}$.

For the derivation of the formula, we need the smallest integer $N$ such that for all $j > N$:

$$e^{-\lambda_j t_k} < \texttt{tol}.$$

With $\lambda_j = (j\pi/L)^2$:

$$e^{-(j\pi/L)^2 t_k} < \texttt{tol} \quad \Leftrightarrow \quad (j\pi/L)^2 t_k > \ln(1/\texttt{tol}) \quad \Leftrightarrow \quad j > \frac{L}{\pi}\sqrt{\frac{\ln(1/\texttt{tol})}{t_k}}.$$

So the boundary (largest $j$ that still needs the full integral) is:

$$j_* = \sqrt{\frac{\ln(1/\texttt{tol})}{(\pi/L)^2\, t_k}}.$$

In the code, $\texttt{eigenvalue_scale} = (\pi/L)^2$, so

$$j_* = \sqrt{\frac{\ln(1/\texttt{tol})}{\texttt{eigenvalue_scale} \cdot t_k}}$$

- Modes with $j \le j_*$: full integral.
- Modes with $j > j_*$: asymptotic.

So we want $N = \lceil j_* \rceil$. Then:

- For $j = 1, \ldots, N$ we do the full integral.
- For $j = N+1, \ldots, n_k$ we use $\hat{F}_j(t_k)/\lambda_j$.

That's exactly
return `math.ceil(math.sqrt(log_inv_tol / (eigenvalue_scale * tk)))`.

with `log_inv_tol = ln(1/tol)`.

$j_*$ is generally not an integer. Using ceil means we round the cutoff up: we keep one more mode in the "full integral" set. So we never switch to the asymptotic approximation too early, and $e^{-\lambda_j t_k}$ is guaranteed to be $\le \texttt{tol}$ for all $j > N$. That keeps the approximation safe (slightly conservative).

Summary

- The cutoff is chosen so that for $j > N$, $e^{-\lambda_j t_k} < \texttt{tol}$.
- Solving $e^{-(j\pi/L)^2 t_k} < \texttt{tol}$ gives $j > \sqrt{\ln(1/\texttt{tol})/((\pi/L)^2 t_k)}$.
- So $N = \lceil \sqrt{\ln(1/\texttt{tol})/((\pi/L)^2 t_k)} \rceil$ is the last mode that gets the full time integral; higher modes use the cheap $\hat{F}_j(t_k)/\lambda_j$ formula.

## 4.3 GPU-batched forcing per $t_k$

The GPU-batched per-$t_k$ approach keeps the *mathematical* structure of $I_j(t_k)$ but replaces repeated scalar quadratures with *batched CuPy operations* on fixed grids in both $x$ and $s$. For a given $t_k$, we:

1. Fix grids:

- $x$-grid: $x_0, \ldots, x_{N_x-1}$ in $[0, L]$,
- $s$-grid: $s_0, \ldots, s_{N_s-1}$ in $[0, t_k]$.

2. Evaluate $F(x, s)$ on the $(s, x)$ grid in a single CuPy call:

$$F_{i\ell} \approx F(x_\ell, s_i), \quad 0 \le i < N_s,\ 0 \le \ell < N_x,$$

   i.e. `F_vals` with shape $(N_s, N_x)$.

3. Precompute $\sin(jx_\ell)$ for all modes $j = 1, \ldots, n_k$ and nodes $x_\ell$:

$$S_{j\ell} = \sin(jx_\ell).$$

4. Form the integrand for the $x$-integral for all $(s_i, j)$ at once:

$$G_{ij\ell} = F_{i\ell}\, S_{j\ell},$$

   and use the trapezoidal rule over $x$ to approximate $\hat{f}_j(s_i)$:

$$\hat{f}_j(s_i) \approx \frac{2}{L} \int_0^L F(x, s_i) \sin(jx)\, dx \approx \frac{2}{L}\ \mathrm{trapz}(G_{ij\ell} \text{ in } x_\ell).$$

5. Form the integrand for the $s$-integral:

$$H_{ij} = \hat{f}_j(s_i)\, \mathrm{e}^{-\lambda_j (t_k - s_i)},$$

   and again use the trapezoidal rule in $s$ to approximate $I_j(t_k)$:

$$I_j(t_k) \approx \int_0^{t_k} \hat{f}_j(s)\, \mathrm{e}^{-\lambda_j (t_k - s)}\, ds \approx \mathrm{trapz}(H_{ij} \text{ in } s_i).$$

6. Finally, sum over modes with $\sin(jx_0)$ to get $u^{(F)}(x_0, t_k)$:

$$u^{(F)}(x_0, t_k) \approx \sum_{j=1}^{n_k} I_j(t_k)\ \sin(jx_0).$$

All of the above is done with batched array ops in CuPy, so the triple loop over $(k, j, s)$ becomes a small number of large GPU kernels per $t_k$.
This is the core idea behind the specialized kernel `compute_forcing_contribution_at_time_default_F_c` and can be generalized to arbitrary CuPy-compatible $F$.

# 5 Per-$t_k$ Batching Strategy

The per-$t_k$ strategy means:

- For each time step $t_k$, we launch a GPU computation that:
  - Integrates in $x$ and $s$ for *all* modes $j = 1, \ldots, n_k$,
  - Returns a scalar $u^{(F)}(x_0, t_k)$.

This has several advantages:

1. Simplicity of integration into the existing algorithm:
   - The CPU code already loops over $k$.
   - We can replace the inner call to `compute_forcing_contribution_at_time` with a call to a GPU routine of the same conceptual signature.
2. Good utilization of the GPU (when $n_k$, $N_x$, and $N_s$ are sufficiently large):
   - Each call performs $O(N_s N_x n_k)$ floating-point operations in massively parallel fashion.
3. Clear separation of concerns:
   - The outer reconstruction algorithm does not need to know the details of the forcing integrals; it only needs $u^{(F)}(x_0, t_k)$.

In the future, a fully batched strategy over all $t_k$ could further improve performance by exploiting shared grids or convolution structure in time, but the per-$t_k$ approach already yields substantial speedups (e.g., the observed $\sim 7.44\times$ improvement) without deeply restructuring the algorithm.

# 6 Summary of the First Main GPU-Batched Implementation

The "first main" GPU-batched implementation for the non-homogeneous reconstruction with per-$t_k$ forcing consists of:

- Homogeneous part on GPU:
  - Use CuPy to compute the homogeneous spectral contribution $u^{(0)}(x_0, t_k)$ from $\hat{f}_j$.
- Forcing part on GPU (per-$t_k$):

– For each $t_k$, approximate

$$u^{(F)}(x_0, t_k) = \sum_{j=1}^{n_k} I_j(t_k)\, \sin(jx_0)$$

via batched CuPy quadrature in $x$ and $s$.

– Use a theoretically justified truncation index $n_k$ from

$$n_k = \left\lceil C\, \mathrm{e}^{\frac{(k+1)^2 t_k}{2}} \right\rceil, \quad C = \frac{2}{L} \max_s \|F_x(\cdot, s)\|_{L^1(0,L)}.$$

- Reconstruction loop on GPU:
  – Combine homogeneous and forcing contributions to form $u_{\mathrm{data}}(t_k) = u^{(0)}(x_0, t_k) + u^{(F)}(x_0, t_k)$,
  – Run the same recursive scheme for $\bar{f}_{\hat{k}}$ on the GPU,
  – Reconstruct $\bar{f}_n(x)$ via a CuPy-based sine series.

This design:

- Preserves the *mathematical structure* and truncation guarantees of the original algorithm,
- Moves the main numerical bottlenecks (nested quadratures and sums) to the GPU using *batched array operations*, and
- Demonstrates substantial speedups for realistic problem sizes while remaining close enough to the original code to be maintainable.